Indexing 🗂

Indexing is a data structure technique used to quickly locate and access data from a database. An index is a small, separate table that stores a copy of a few columns of a database table, along with a pointer to the address of the corresponding record in the main table. By using an index, the database system can avoid a full scan of the entire table, making data retrieval much faster.

Think of an index like the index at the back of a book. Instead of reading the entire book to find a topic, you go to the index, find the topic and its corresponding page number, and go directly to that page. Similarly, a database index points the system directly to the location of the data on disk.

Types of Indexing

There are two primary types of indexing: **ordered indices** and **hash indices**.

1. Ordered Indices

In an ordered index, the entries are stored in a sorted order. This makes it efficient for range searches (e.g., finding all records with a value between 50 and 100).

- **Primary Index**: An index on the **primary key** of a table. A primary index is a sparse index, meaning it only stores an entry for some of the blocks in the main table. This is because the primary key is unique and sorted, so there's no need to store a pointer for every single record.
- **Secondary Index**: An index on a non-key attribute. Secondary indices are dense, meaning they contain an entry for **every record** in the data file. This is necessary because the non-key attribute may have duplicate values, and we need to be able to find all records with a particular value.

2. Hash Indices

A hash index uses a **hashing function** to map a key to a specific location in a hash table. This is extremely fast for equality searches (e.g., finding a record where the value is exactly 50).

- **How it works**: A hash index computes a hash value from a key and uses it to locate the corresponding record. This provides **O(1)** (constant time) average-case performance for retrieval.

Indexing on Multiple Columns

An index can also be created on multiple columns. This is useful when queries frequently involve filtering by more than one attribute.

- **Composite Index**: An index created on a combination of two or more columns. The order of the columns in a composite index is crucial, as the database uses a left-to-right approach for

searching. For example, an index on `(LastName, FirstName)` would be efficient for queries on `LastName` or `LastName` and `FirstName` together, but not for a query on `FirstName` alone.

Advantages and Disadvantages of Indexing

Advantages	Disadvantages
Faster Data Retrieval: Queries that use indexed columns are much faster.	**Slower Data Modification**: Indexing can slow down insertion, deletion, and update operations. Every time data in the main table is modified, the index must also be updated.
Improved Query Performance: Improves the performance of `WHERE`, `ORDER BY`, `GROUP BY`, and `JOIN` clauses.	**Storage Overhead**: Indices consume disk space. The more indices you have, the more storage is required.
Reduced I/O Operations: By accessing a smaller index, the system reduces the number of disk I/O operations.	**Maintenance**: Indices need to be maintained and can become fragmented over time, requiring periodic reorganization.